

Report on SAT Solving Exercise

DA7067 Computational Mathematics

Onyinyechi Sonia Bäckström
September 2026

Department of Mathematics
Stockholm University

Introduction

The purpose of this exercise is to use Boolean satisfiability (SAT) solving to solve two graph problems. SAT solving provides a general and powerful framework for determining whether a Boolean formula has a satisfying assignment.

This report presents SAT-based solutions to two graph problems using the PySAT library. The first problem concerns determining whether an undirected graph can be colored using at most k colors such that no two adjacent vertices have the same color. The problem is reduced to Boolean satisfiability by introducing Boolean variables representing vertex-color assignments and constructing a CNF formula.

The second problem concerns finding the smallest set of vertices W whose removal makes an undirected graph 2-colorable. A decision version of this problem is formulated using three possible states for each vertex: red, green, and removed. An AtMostK cardinality constraint limits the number of removed vertices. The decision procedure is then repeatedly used to solve the minimization problem.

Tests were performed on graphs of different sizes and edge densities, as well as using different values of k . The results show how graph size and structure affect SAT-solving time and how repeated decision-solver calls influence the efficiency of the vertex-removal minimization procedure.

Problem 1

This problem concerns solving graph coloring by reducing it to SAT. Using PySAT, a SAT solver can then be used to determine whether a graph can be colored with at most k colors such that no two adjacent vertices have the same color.

Given an undirected graph $G = (V, E)$ and a positive integer k , the objective is to determine whether the graph can be colored using at most k colors such that adjacent vertices have different colors.

The problem is encoded as a CNF formula, which is then given to a SAT solver.

The following function reads a graph from a file.

```
#Read the graph
#satgraph as filename

def read_graph(satgraph):
    """
    Read the graph.

    Returns:
        n      - number of vertices
        edges  - list of edges
    """

    with open(satgraph, "r") as f:
        lines = [line.strip() for line in f if line.strip()]

    n, m = map(int, lines[0].split())

    edges = []

    for line in lines[1:]:
        values = list(map(int, line.split()))

        v = values[0]

        for w in values[1:]:
            edges.append((v, w))

    return n, edges
```

The `read_graph` function reads the number of vertices and edges from the input file and stores the edges as pairs of vertices.

Problem 1a

For each vertex and color, a Boolean variable is introduced. The function maps each vertex-color pair to a unique SAT variable.

For example, if $k = 3$, vertex 1 has variables corresponding to colors 1, 2 and 3, while vertex 2 has the next three variables.

A variable is True when the corresponding vertex is assigned that color.

```
#Create SAT variables/Literals

def color_variable(vertex, color, k):
    """
    Give each (vertex, color) pair a unique positive integer.
    """

    return (vertex - 1) * k + color
```

The following function constructs the CNF formula.

```

#build the CNF formula
def graph_coloring_cnf(satgraph, k):
    """
    Convert graph k-coloring into a CNF formula.

    Returns:
        CNF formula
    """
    n, edges = read_graph(satgraph)
    cnf = CNF()
    # Each vertex must have at least one color
    for v in range(1, n + 1):
        clause = []
        for c in range(1, k + 1):
            clause.append(color_variable(v, c, k))
        cnf.append(clause)
    # Each vertex can have at most one color
    for v in range(1, n + 1):
        for c1 in range(1, k + 1):
            for c2 in range(c1 + 1, k + 1):
                x1 = color_variable(v, c1, k)
                x2 = color_variable(v, c2, k)

                cnf.append([-x1, -x2])
    # Adjacent vertices cannot have the same color
    for u, v in edges:
        for c in range(1, k + 1):
            x_u = color_variable(u, c, k)
            x_v = color_variable(v, c, k)
            cnf.append([-x_u, -x_v])
    return cnf

```

There are three types of constraints.

Constraint 1: Every vertex has a color

For every vertex v , $x_{v,1} \vee x_{v,2} \vee \dots \vee x_{v,k}$. This ensures that every vertex receives at least one color.

Constraint 2: A vertex cannot have two colors

For every pair of colors c_1, c_2 , $(\neg x_{v,c_1} \vee \neg x_{v,c_2})$. Together with the first constraint, this means every vertex receives exactly one color.

Constraint 3: Adjacent vertices cannot have the same color

For every edge (u, v) and color c , $(\neg x_{u,c} \vee \neg x_{v,c})$. Therefore, two adjacent vertices cannot be assigned the same color.

Problem 1b

The solver used in the tests is **Glucose3**.

The function returns:

- True if the graph can be colored using at most k colors.
- False if the graph cannot be colored using at most k colors.

The following graph was used to check the implementation:

```

4 4
1 2 3
2 3
3 4

```

```
#Call the solver
#use PySAT to call a SAT solver to decide whether the formula is satisfiable.
def can_color(satgraph, k):

    cnf = graph_coloring_cnf(satgraph, k)

    with Solver(name="glucose3", bootstrap_with=cnf.clauses) as solver:

        return solver.solve()
```

[14]:

```
#Test the CNF
cnf2 = graph_coloring_cnf("graph.txt", 2)
print("k=2:", cnf2.nv, len(cnf2.clauses), can_color("graph.txt", 2))

cnf3 = graph_coloring_cnf("graph.txt", 3)
print("k=3:", cnf3.nv, len(cnf3.clauses), can_color("graph.txt", 3))
```

k=2: 8 16 False

k=3: 12 28 True

This graph has four vertices and four edges.

k	Result	Interpretation
1	False	The graph is not 1-colorable
2	False	The graph is not 2-colorable
3	True	The graph is 3-colorable

For $k = 2$, the CNF contains 8 Boolean variables and 16 clauses. For $k = 3$, the CNF contains 12 Boolean variables and 28 clauses.

The results show that the graph cannot be colored with 1 or 2 colors, since the solver returns False for $k = 1$ and $k = 2$. When $k = 3$, the solver returns True, meaning that the graph can be colored using at most 3 colors.

Problem 1c

The purpose of this part was to investigate how the SAT solver behaves when the graph size, number of edges, and number of colors are changed.

Runtime was measured using Python's `time.perf_counter()`.

Test 1- Changing graph size

The table above is the result of the test using changing graph size and number of colors fixed at four.

Graph	Vertices	Edges	Colors	SAT	Runtime (s)
vertices_10.txt	10	18	4	True	0.011847
vertices_20.txt	20	97	4	False	0.002087
vertices_30.txt	30	233	4	False	0.001915
vertices_40.txt	40	363	4	False	0.002674
vertices_50.txt	50	586	4	False	0.003949

For this test, the number of vertices and edges increased while the number of colors was fixed at four. The runtime did not increase monotonically with graph size: the 10-vertex graph had the highest measured runtime, while the 20- and 30-vertex graphs were faster. However, from 30 to 50 vertices,

the runtime generally increased as the CNF became larger. The 10-vertex graph was satisfiable with four colors, whereas the larger graphs were unsatisfiable. These results show that SAT-solver runtime depends not only on graph size but also on the structure of the CNF formula and whether the instance is satisfiable or unsatisfiable.

I also tested graphs with increasing numbers of vertices and edges while keeping the number of colors fixed at three. Below table is the result.

Graph	Vertices	Edges	Colors	SAT	Runtime (s)
vertices_10.txt	10	18	3	False	0.000714
vertices_20.txt	20	97	3	False	0.001413
vertices_30.txt	30	233	3	False	0.002315
vertices_40.txt	40	363	3	False	0.002114
vertices_50.txt	50	586	3	False	0.002971

For this test, the number of colors was fixed at three while the number of vertices and edges increased. All graphs were unsatisfiable with three colors. The runtime generally increased as the graph became larger, although it did not increase monotonically. This shows that larger graphs and their resulting CNF formulas can require more computational work, but SAT-solver runtime is not strictly monotonic.

I repeated the experiment with the number of colors fixed at ten while increasing the number of vertices and edges.

Graph	Vertices	Edges	Colors	SAT	Runtime (s)
vertices_10.txt	10	18	10	True	0.001457
vertices_20.txt	20	97	10	True	0.003725
vertices_30.txt	30	233	10	True	0.005572
vertices_40.txt	40	363	10	True	0.033381
vertices_50.txt	50	586	10	True	0.017265

In this test, the number of colors was fixed at 10 while the number of vertices and edges increased. All graphs were satisfiable because 10 colors were sufficient for these instances. The runtime generally increased as the graph size increased, although it was not strictly monotonic, with the 40-vertex graph having the highest runtime. This shows that larger CNF formulas can require more computational work, but the runtime also is not strictly monotonic.

Test 2 – Changing Edge Density

The number of vertices was fixed at 20 and the probability of generating an edge was varied.

Graph	Edge probability	Vertices	Actual edges	Colors	SAT	Runtime (s)
Density_10.txt	10%	20	18	4	True	0.001163
Density_30.txt	30%	20	61	4	True	0.002597
Density_50.txt	50%	20	99	4	False	0.001960
Density_70.txt	70%	20	143	4	False	0.001341
Density_90.txt	90%	20	172	4	False	0.001307

From the table above, increasing the edge density increases the size of the CNF formula because each additional edge introduces further constraints. The graphs with 10% and 30% edge density were satisfiable with four colors, while the denser graphs were unsatisfiable. The runtime did not increase

monotonically with edge density. The 30% graph had the highest runtime, while the densest graphs were solved faster. This shows that SAT-solver performance depends not only on the number of clauses but also on the structure of the formula and whether a satisfying assignment can be found or unsatisfiability can be established quickly.

Test 3 – Changing number of colors

A fixed graph containing 20 vertices and 60 edges was used. The table below is the result.

Colors k	Vertices	Edges	Edge Probability	SAT	Runtime (s)
2	20	60	30%	False	0.001293
3	20	60	30%	False	0.001855
4	20	60	30%	True	0.001348
5	20	60	30%	True	0.001145

The color test shows that increasing k does not necessarily lead to higher runtime. For the fixed 20-vertex graph, $k = 2$ and $k = 3$ were unsatisfiable, while $k = 4$ and $k = 5$ were satisfiable. The runtimes varied rather than increasing consistently.

The graph could not be colored using two or three colors but could be colored using four and five colors. Increasing the number of colors has two competing effects. First, more colors produce more Boolean variables and more color-related clauses. However, increasing the number of available colors also makes the coloring problem less restrictive.

In conclusion, the three tests show that the performance of the SAT solver depends on several characteristics of the graph and the CNF formula. Increasing the graph size generally increased the runtime, although the increase was not strictly monotonic. Increasing edge density also produced larger CNF formulas, but runtime did not consistently increase because it depends on the structure and satisfiability of the instance. Similarly, increasing the number of colors makes the CNF larger but also makes the coloring problem less restrictive, so the runtime does not necessarily increase. Overall, the results show that graph size, edge density, number of colors and the SAT/UNSAT status all influence solver performance.

Problem 2

The second problem asks for the smallest set of vertices $W \subseteq V$ such that removing these vertices makes the remaining graph $G - W$ 2-colorable.

The SAT formulation uses three states for each vertex: R (red), G (green) and S (removed). Each vertex receives exactly one state, and S-vertices may be adjacent to any vertices because they are ultimately removed. The implementation uses an AtMostK cardinality constraint on the S- variables.

Problem 2a

The image below is the SAT reduction and CNF construction.

```
#SAT reduction

def removal_cnf(satgraph, k):

    n, edges = read_graph(satgraph)

    cnf = CNF()

    # Each vertex gets exactly one of:
    # R = red
    # G = green
    # S = removed

    for v in range(1, n + 1):

        R = removal_variable(v, 1)
        G = removal_variable(v, 2)
        S = removal_variable(v, 3)

        # At Least one color
        cnf.append([R, G, S])

        # At most one color
        cnf.append([-R, -G])
        cnf.append([-R, -S])
        cnf.append([-G, -S])

    # Adjacent vertices cannot have the same
    # red or green color.

    for u, v in edges:

        R_u = removal_variable(u, 1)
        G_u = removal_variable(u, 2)

        R_v = removal_variable(v, 1)
        G_v = removal_variable(v, 2)

        # u and v cannot both be red
        cnf.append([-R_u, -R_v])

        # u and v cannot both be green
        cnf.append([-G_u, -G_v])

    # At most k vertices can be removed

    star_variables = []

    for v in range(1, n + 1):
        star_variables.append(removal_variable(v, 3))

    #At most k vertices maybe removed

    cardinality = CardEnc.atmost(
        lits=star_variables,
        bound=k,
        top_id=3 * n
    )

    cnf.extend(cardinality.clauses)

    return cnf
```

Each vertex has three Boolean variables representing its possible states. The first group of clauses ensures that each vertex has exactly one state. The edge constraints ensure that adjacent vertices which remain in the graph cannot have the same color.

The final cardinality constraint ensures that no more than k vertices are assigned the removed state.

Decision Solver: Below is the decision procedure

```
#Decision solver problem

def can_remove_k(satgraph, k):

    cnf = removal_cnf(satgraph, k)

    with Solver(
        name="glucose3",
        bootstrap_with=cnf.clauses
    ) as solver:

        return solver.solve()
```

The following function extracts the vertices assigned the removed state, W .

```
def get_removed_vertices(satgraph, k):

    cnf = removal_cnf(satgraph, k)

    with Solver(
        name="glucose3",
        bootstrap_with=cnf.clauses
    ) as solver:

        if not solver.solve():
            return None

        model = set(solver.get_model())

    n, edges = read_graph(satgraph)

    W = []

    for v in range(1, n + 1):

        S = removal_variable(v, 3)

        if S in model:
            W.append(v)

    return W
```

The tables below are the results of the decision problem for graph.txt and two_triangle.txt respectively with the set of W that can be removed.

k	Decision	W
0	False	None
1	True	[2]
2	True	[3]
3	True	[1]
4	True	[1]

k	Decision	W
0	False	None
1	False	None
2	True	[2, 5]
3	True	[2, 4]
4	True	[3, 4]

The tables show the results of testing different values of k , where k represents the maximum number of vertices that can be removed.

- For graph.txt, $k = 0$ is False, but $k = 1$ is True. Therefore, the minimum number of vertices that need to be removed is 1. One valid solution is removing vertex 2.

- For two_triangle.txt, both $k = 0$ and $k = 1$ are False, while $k = 2$ is True. Therefore, the minimum number of vertices that need to be removed is 2. One valid solution is removing vertices 2 and 5.

Overall, the results demonstrate that increasing k eventually makes the problem satisfiable, and the first k that gives True represents the minimum number of removals required.

About the efficiency for different inputs, decision problem becomes satisfiable when k reaches the minimum number of vertices that must be removed. For graph.txt, the first True result occurs at $k = 1$, while for two_triangles.txt, it occurs at $k = 2$. Thus, graphs requiring more vertex removals require testing more values of k , making the minimization process less efficient.

Problem 2b

The minimization function is:

```
def minimum_removed_vertices(satgraph):

    n, edges = read_graph(satgraph)

    number_of_calls = 0

    for k in range(n + 1):

        number_of_calls += 1

        if can_remove_k(satgraph, k):

            W = get_removed_vertices(satgraph, k)

            return k, W, number_of_calls
```

The minimization problem can be solved by repeatedly using the decision solver. Starting with $k = 0$, the decision solver is called to determine whether a valid solution exists using at most k removed vertices. If the result is False, k is increased by one and the process is repeated until the first True result is obtained. This first satisfiable k is the minimum number of vertices that must be removed. The corresponding set W is then obtained using an additional SAT solver call. In the worst case, $n + 1$ decision solver calls are required. For the three tested graphs, the numbers of decision calls are 2, 1, and 3, giving an average of $(2 + 1 + 3)/3 = 2$ decision solver calls.

Graph Name	Minimum k	Removed Vertices W	Decision SAT calls
graph.txt	1	[2]	2
path.txt	0	[]	1
two_triangles.txt	2	[2, 5]	3

The table above shows the minimum number of vertices that need to be removed for each graph and the corresponding set W . The number of decision SAT calls ranges from 1 to 3, with an average of 2 calls across the three graphs.

Problem 2c

The minimization problem is solved by testing $k = 0, 1, 2, \dots$ until the first satisfiable instance is found. If the first satisfiable value is k^* , then k^* is the minimum number of vertices that need to be removed. With this linear search strategy, $k^* + 1$ decision-solver calls are required, since the search starts at $k = 0$. In the worst case, this requires $n + 1$ decision-solver calls.

```
def test_minimization(satgraph):
    start = time.perf_counter()

    k, W, calls = minimum_removed_vertices(satgraph)

    end = time.perf_counter()

    vertices, edges = read_graph(satgraph)

    print("Graph:", satgraph)
    print("Vertices:", vertices)
    print("Edges:", len(edges))
    print("Minimum number of removed vertices:", k)
    print("Removed vertices W:", W)
    print("Number of Decision SAT calls:", calls)
    print("Runtime:", end - start, "seconds")
```

The table below summarizes the minimization results for eight different graphs. It shows the minimum number of vertices that must be removed, the corresponding set W , the number of decision-solver calls required and runtime.

Graph	Vertices	Edges	Minimum W	W	Decision SAT calls	Runtime (s)
graph.txt	4	4	1	[2]	2	0.002013
path.txt	4	3	0	[]	1	0.001505
two_triangle.txt	6	8	2	[2, 5]	3	0.002695
larger.txt	8	13	1	[3]	2	0.002210
large_10.txt	10	8	0	[]	1	0.001677
large_20.txt	20	67	8	[2, 6, 10, 11, 13, 14, 18, 19]	7	0.02267
large_30.txt	30	123	12	[1, 2, 3, 4, 6, 8, 14, 23, 24, 25, 27, 30]	13	0.117490
large_40.txt	40	233	21	[2, 3, 4, 6, 9, 12, 13, 16, 17, 24, 25, 28, 29, 30, 32, 33, 36, 38, 39]	22	0.880835

The table shows that the minimization solver generally becomes increasingly computationally demanding as the graph size increases. For the larger graphs, the minimum number of removed vertices increases from 8 for the 20-vertex graph, to 12 for the 30-vertex graph, and 21 for the 40-vertex graph. Correspondingly, the number of decision-SAT calls increases from 7 to 13 to 22. The runtime also increases significantly, from 0.02267 s for 20 vertices to 0.11749 s for 30 vertices and 0.880835 s for 40 vertices. The smaller graphs require fewer calls and much less time.

Overall, the results show that larger and more complex graphs tend to require more vertex removals, more decision-solver calls, and longer runtimes. This demonstrates that the repeated decision-solving approach becomes increasingly expensive as the size and difficulty of the graph increase.

The efficiency of the minimization solver depends strongly on the size and complexity of the graph. Smaller graphs require fewer decision-SAT calls and have very short runtimes, while larger graphs require more calls and significantly more time. For example, the 20-vertex graph requires 7 calls and 0.02267 seconds, compared with 13 calls and 0.11749 seconds for the 30-vertex graph, and 22 calls and 0.880835 seconds for the 40-vertex graph. This shows that the repeated decision-solving approach becomes increasingly computationally demanding for larger graphs.

Conclusion

This exercise showed how SAT solving can be used to formulate and solve two different graph problems using the PySAT library.

For Problem 1, graph coloring was successfully reduced to a CNF formula using Boolean variables representing vertex-color assignments. The SAT solver was then able to determine whether a graph could be colored using a specified number of colors. The tests showed that solver runtime was affected by several factors, including the number of vertices, number of edges, number of colors, and whether the resulting SAT instance was satisfiable or unsatisfiable. However, the runtime did not consistently increase with graph size or edge density. In several cases, larger or denser instances were solved faster than smaller instances. This indicates that the number of variables and clauses alone is not sufficient to predict SAT-solving time; the structure and difficulty of the resulting CNF formula also play an important role.

For Problem 2, the vertex-removal problem was formulated as a SAT decision problem using three possible states for each vertex: red, green, and removed. An AtMostK constraint was used to limit the number of removed vertices. The minimization problem was then solved by repeatedly increasing the value of k until the first satisfiable instance was found. The tests' results showed that the larger test graphs required more decision calls and generally had longer runtimes. In particular, the 40-vertex example required substantially more time than the smaller examples.

